

Name : Anup Rajesh Yadav

PRN : 202501050062

Batch : CH3

Subject : EDS

Screenshot Of Practical No.4 :

Practical Lab Assignment :

The image displays two screenshots of the CODETANTRA IDE interface, showing practical assignments related to Pandas data manipulation.

Top Screenshot: 4.1.1. Pandas - series creation and manipulation

The task description states: "Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations." A hint suggests grouping by even/odd status. The input format requires a space-separated list of numbers. The output format requires displaying the mean for even and odd numbers separately, with labels indicating the corresponding group.

The code in the editor is as follows:

```
1 import pandas as pd
2
3 # Take inputs from the user to create a list of numbers
4 numbers = list(map(int, input().split()))
5
6
7 # Create a Pandas series from the list of numbers
8 s = pd.Series(numbers)
9
10 # Grouping by even and odd numbers and calculating the mean
11 # Grouping by even and odd numbers and calculating the mean
12 grouped = s.groupby(s % 2 == 0).mean()
13
14 # Display the mean of even and odd numbers with labels
15 grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
16 print("Mean of even and odd numbers:")
```

The test results show 3 out of 3 shown test case(s) passed and 3 out of 3 hidden test case(s) passed. The test case 1 output shows the expected output as a list of means: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10].

Bottom Screenshot: 4.1.2. Dictionary to dataframe

The task description states: "A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation." The operations listed are: Create the DataFrame (convert dictionary to Pandas DataFrame), Add a new row (take inputs for name and age, add to DataFrame, display), Modify a row (modify age by row index, display), Delete a row (remove by row index, display), Add a new column (add 'Gender' column, display), Modify a column (convert names to uppercase, display), and Delete a column (remove 'Age' column, display).

The code in the editor is as follows:

```
1 # import pandas as pd
2
3 # Provided dictionary of lists
4 data = {
5     'Name': ['Alice', 'Bob', 'Charlie'],
6     'Age': [25, 30, 35],
7 }
8
9 # Convert the dictionary to a DataFrame
10 df = pd.DataFrame(data)
11
12 # Display the original DataFrame
13 print("Original DataFrame:")
14 print(df)
15
16 # Adding a new row
```

The test results show 1 out of 1 shown test case(s) passed and 1 out of 1 hidden test case(s) passed. The test case 1 output shows the original DataFrame with columns Name and Age, and the new DataFrame with an added 'Gender' column.

4.1.3. Student Information

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:
Refer to the displayed test cases for better understanding.

Sample Test Cases

studentin...

```
1 import pandas as pd
2
3 # Read the text file into a DataFrame
4 file = input()
5 data = pd.read_csv(file, sep="\t+", header=None, names=["Name", "Age", "Grade"])
6
7
8 # write your code here..
9 print("First five rows:")
10 print(data.head())
11
12 avg_age = round(data["Age"].mean(), 2)
13 print("Average age:", avg_age)
14
15 filtered_data = data[data["Grade"].isin(["A", "B"])]
16
17 print("Students with a grade on A or B")
```

Average time
0.021 s
21.00 ms

Maximum time
0.032 s
32.00 ms

1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed

Test case 1

Expected output

Actual output

studentdata.txt

studentdata.txt

First five rows:

First five rows:

...

...

0: John ... 25 ... A

0: John ... 25 ... A

1: Allen ... 22 ... B

1: Allen ... 22 ... B

2: Emma ... 24 ... A

2: Emma ... 24 ... A

Terminal

Test cases

< Prev

Reset

Submit

Next >

Lab Assignment :

CODETANTRA

Home Learn Anywhere

202501050062@mitaoe.ac.in Support Logout

4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the columns: Date, Product, Quantity, Price, and City.

Group the data by Month and calculate the total sales for each month.

Find the month with the highest total sales and display it.

Also, display the total sales for the best month.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

1 import pandas as pd

2

3 # Prompt the user for the file name

4 file_name = input()

5

6 # Load the data

7 df = pd.read_csv(file_name)

8 df["Date"] = pd.to_datetime(df["Date"])

9 df["Month"] = df["Date"].dt.strftime("%Y-%m")

10 df["Total Sales"] = df["Quantity"] * df["Price"]

11

12 # Find the month with the highest total sales

13 sales_by_month = df.groupby("Month")["Total Sales"].sum()

14 best_month = sales_by_month.idxmax()

15 highest_sales = sales_by_month.max()

16

17 return f"Best month: {best_month}\nTotal sales: {highest_sales}"

Average time 0.015 s

Maximum time 0.030 s

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

Expected output sales_data.csv

Best month: 2025-01

Total sales: \$1210.00

Actual output sales_data.csv

Best month: 2025-01

Total sales: \$1210.00

Terminal

Test cases

CODETANTRA

Home Learn Anywhere

202501050062@mitaoe.ac.in Support Logout

4.2.2. Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the columns: Date, Product, Quantity, Price, and City.

Find the product that sold the most in terms of quantity sold.

Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

1 import pandas as pd

2

3 # Prompt the user for the file name

4 file_name = input()

5

6 # Load the data

7 df = pd.read_csv(file_name)

8

9 product_sales = df.groupby("Product")["Quantity"].sum()

10 # Find the product with the highest total quantity sold

11 best_product = product_sales.idxmax()

12 highest_quantity = product_sales.max()

13

14 # Display the result

15 print(f"Best selling product: {best_product}")

16 print(f"Total quantity sold: {highest_quantity}")

17

Average time 0.012 s

Maximum time 0.023 s

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

Expected output sales_data.csv

Best selling product: Product A

Total quantity sold: 23

Actual output sales_data.csv

Best selling product: Product A

Total quantity sold: 23

Terminal

Test cases

Prev

Reset

Submit

Next

CODETANTRAHomeLearn Anywhere202501050062@mitaoe.ac.inSupportLogout

4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by city and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 # write the code..
10 city_sales = df.groupby("City")["Quantity"].sum()
11 best_city=city_sales.idxmax()
12 # Display the result
13 print(f"city sold the most products: {best_city}")
14
```

Average time0.010 sMaximum time0.021 s1 out of 1 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1Expected outputActual outputCity sold the most products: Los AngelesCity sold the most products: Los Angeles

CODETANTRAHomeLearn Anywhere202501050062@mitaoe.ac.inSupportLogout

4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pairs that were sold most frequently.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

- Product A and Product B: Appear in transactions on 2025-01-01 and 2025-01-03.
- Product A and Product C: Appear in transactions on 2025-01-02 and 2025-01-05.
- Product B and Product C: Appear in transactions on 2025-01-04.

Most Frequent Product Combinations:

- Product A and Product B (2 times)
- Product A and Product C (2 times)

frequent...

```
1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11 # write the code
12 date_products = {}
13
14 for date, group in df.groupby("Date"):
15     products = group['Product'].unique()
16     if len(products) > 1:
17         date_products[date] = products
```

Average time0.011 sMaximum time0.024 s1 out of 1 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1Expected outputActual outputProduct A and Product B: 2 timesProduct A and Product B: 2 timesProduct A and Product C: 2 timesProduct A and Product C: 2 times

CODETANTRAHomeLearn Anywhere202501050062@mitaoe.ac.inSupportLogout

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

- Display the first 5 rows of the dataset.
- Display the last 5 rows of the dataset.
- Get the shape of the dataset (number of rows and columns).
- Get a summary of the dataset (using .info()).
- Get basic statistics (mean, standard deviation, etc.) of the dataset using .describe().
- Check for missing values and display the count of missing values for each column.
- Fill missing values in the 'Age' column with the median age.
- Fill missing values in the 'Embarked' column with the most frequent value (mode()).
- Drop the 'Cabin' column due to many missing values.
- Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

Passenger Id	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25	5		
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833	C85	C	
3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2	31.00	282	7.925	S
4	1	1	Tuttle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1	C123	S	
5	0	3	Allen, Mr. William Henry	male	35	0	0	373459	8.05	5		
6	0	3	Moran, Mr. James	male	0	0	0	330877	8.45	1		

titanicDat...

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 import pandas as pd
8 import numpy as np
9
10 # Load the Titanic dataset
11 data = pd.read_csv('Titanic-Dataset.csv')
12
13 # 1. Display the first 5 rows of the dataset
14 print(data.head())
15
16 # 2. Display the last 5 rows of the dataset
17 print(data.tail())
```

Average time0.0092 sMaximum time0.092 s1 out of 1 shown test case(s) passed

Test case 1Expected outputActual outputPassengerId Survived Pclass Fare Cabin Embarked0 0 3 7.2500 NaN 51 1 71.2833 C85 1 71.2833 C85 2 1 31.00 282 7.925 S3 1 53.1 123 NaN 5

202501050062@mitaoe.ac.inSupportLogout

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.

2. Convert the 'Sex' column to numeric values (male: 0, female: 1).

3. One-hot encode the 'Embarked' column, dropping the first category.

4. Get the mean age of passengers.

5. Get the median fare of passengers.

6. Get the number of passengers by class.

7. Get the number of passengers by gender.

8. Get the number of passengers by survival status.

9. Calculate the survival rate of passengers.

10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

Passenger Id	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,S
6,0,3,"Moran, Mr. James",male,0,0,330877,8.4583,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,S

202501050062@mitaoe.ac.inSupportLogout

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.

2. Calculate the survival rate by embarkation location (Embarked_S).

3. Calculate the survival rate by family size (FamilySize).

4. Calculate the survival rate by being alone (IsAlone).

5. Get the average fare by passenger class (Pclass).

6. Get the average age by passenger class (Pclass).

7. Get the average age by survival status (Survived).

8. Get the average fare by survival status (Survived).

9. Get the number of survivors by class (Pclass).

10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

Passenger Id	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,S
6,0,3,"Moran, Mr. James",male,0,0,330877,8.4583,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

202501050062@mitaoe.ac.inSupportLogout

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).

2. Get the number of non-survivors by gender (Sex).

3. Get the number of survivors by embarkation location (Embarked_S).

4. Get the number of non-survivors by embarkation location (Embarked_S).

5. Calculate the percentage of children (Age < 18) who survived.

6. Calculate the percentage of adults (Age >= 18) who survived.

7. Get the median age of survivors.

8. Get the median age of non-survivors.

9. Get the median fare of survivors.

10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

Passenger Id	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,S
6,0,3,"Moran, Mr. James",male,0,0,330877,8.4583,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

